Take-Home: Signal Sifter

Brainify Technology — Software Engineering Internship

Time budget: **~5 hours.** Deadline: **Tuesday 1 September 2026, 23:59 (BST)**

Before you start — please read this part

This task is deliberately **not** a test of what you already know.

Most of you have told us on your CV that you work in React and Node. This task is in Python, and it asks you to use a library you have probably never touched. That is on purpose. We are far more interested in **how you approach something new** than in watching you rebuild something you have built before.

Three things follow from that:

1. **You are not expected to know Python already.** If this is your first Python program, say so in your `DECISIONS.md` and tell us how you got moving. We read that, and it counts in your favour — not against you.
2. **Five hours means five hours.** We would much rather see a smaller thing done thoughtfully, with a clear note about what you left out, than a rushed attempt at everything. Finishing every requirement is *not* the goal. Good judgement about what to cut *is* part of what we are assessing.
3. **Ask us questions.** Email **career@brainify.tech** any time. Asking questions does not count against you, and "I asked nothing" is not a virtue here. If something in this brief seems unclear, incomplete, or contradictory, we would like to hear about it.

You may use AI tools — ChatGPT, Claude, Copilot, Cursor, whatever you normally use. We use them too. We only ask that you can explain everything you submit, and that you tell us in `DECISIONS.md` where you used them and what you had to fix. Using AI well is a skill we value. Submitting code you do not understand is not.

The problem

We run crawlers that collect plugin and product listings from marketplaces. The raw output is messy: it comes from several generations of scraper code, so field names disagree, dates arrive in whatever format the source used, the same product shows up more than once, and some files are simply broken.

Somebody downstream needs a clean, ranked shortlist.

You are given **40 raw JSON files** in `data/`. Build a command-line program that turns them into a ranked report.

What it must do

1. **Ingest** every record in `data/`.
2. **Normalise** them into one consistent internal shape.
3. **Deduplicate** records that refer to the same product.

4. **Score** each product using the formula below.
5. **Emit** a ranked report as both JSON and human-readable text.

Running it

```
python -m signal_sifter --input data/ --output report
```

You may change this interface. If you do, document it in your README.

The data

Every file in `data/` is one product listing. They are not consistent with each other. A few things you will notice:

- The product name might be under `title`, `name`, or `product_name`.
- Install counts might be under `installs`, `install_count`, or `active_installs`, and might be a number or a string like `"18,300"`.
- Ratings might be under `rating`, `stars`, or `score`.
- Review counts might be under `reviews`, `review_count`, or `num_ratings`.
- Dates might be under `last_updated`, `updated_at`, or `modified`, and might be ISO 8601, a plain date, a slash-separated date, or a Unix timestamp.
- Some records nest their numbers under a `stats` object instead of at the top level.
- Some fields are missing. Some are `null`.

Handle what you reasonably can. Where you have to make a judgement call, make it, and write down what you decided and why.

Scoring

Treat **2026-09-01** as today's date. (Pinning this keeps everyone's results comparable — do not use the real current date.)

This is a fixed reference point for scoring the supplied data. It is not related to the submission deadline, and nothing about it changes if you submit earlier.

```
installs_points = min(installs / 1000, 50)
rating_points   = (rating - 3.0) * 10
review_points   = min(review_count / 100, 20)

score = installs_points + rating_points + review_points
```

Note that `rating_points` can be negative for poorly-rated products. That is intended.

Staleness rule: any record whose last-updated date is more than 90 days before 2026-09-01 receives a total score of **0**.

The report

The report must contain **exactly the 10 highest-scoring products**, ranked.

At least 4 of the 10 must have a rating above 4.5.

For each entry, show: rank, product name, score, install count, rating, review count, and last-updated date.

The JSON report should be machine-readable. The text report should be something a non-technical colleague could read without asking you what it means.

Required: pydantic

Use **pydantic** to define and validate your normalised data model.

Most candidates will not have used it before. That is fine and expected — the documentation is good, and working out an unfamiliar library from its docs is a normal part of this job. Budget time for it.

Everything else — project layout, CLI library, test framework, how you structure the code — is your choice.

Testing

Include tests. We are looking for a handful of tests that would actually catch a regression, not a large number of shallow ones. If you ran short on time and wrote fewer than you would like, say so in `DECISIONS.md`.

What to submit

A **GitHub repository** (public, or private with `BrainifyTech` invited) containing:

```
├── signal_sifter/       (or whatever you name your package)
├── tests/
├── data/                (the files we gave you, unchanged)
├── README.md
├── DECISIONS.md
└── requirements.txt     (or pyproject.toml)
```

`README.md`

- What it does and how to run it
- How to install dependencies
- How to run the tests
- Anything we need to know to get it working on a clean machine

`DECISIONS.md`

This one matters as much as the code. Please cover:

- **Hours actually spent.** Be honest — over or under, we are not scoring you on hitting five hours exactly. We are checking that you can estimate and report your own work truthfully.
- **What you decided, and why.** Especially anywhere the brief left you a choice.
- **What you left out**, and what you would do next with more time.
- **Anything about this brief that was unclear, incomplete, or contradictory** — and what you did about it.
- **Where you used AI**, what it got wrong, and what you changed.
- **What was new to you here**, and how you got up to speed.
- **Anything you did that we did not ask for**, and why you thought it was worth the time.

Committing

Commit as you work. We read commit history, and we would rather see fifteen honest commits than one large one at the end. Do not commit `.env` files, API keys, credentials, or virtual environment / cache directories.

If you run out of time, **submit what you have with a note** in `DECISIONS.md`. Partial work submitted on time beats complete work submitted late.

How we will assess this

Openly, so there are no surprises. We score five things:

	WHAT WE ARE LOOKING AT
Foundations	Is the data modelled sensibly? Does it handle bad input without falling over? Is the code something another person could work in?
Judgement	Given five hours, did you spend them on the things that mattered?
Learning	How did you approach the parts that were new to you? We compare you against where you started, not against someone who already knew Python.
Initiative	Did you notice the things we did not spell out? Did you ask? Did you make and document decisions rather than guessing silently?
Discipline	Commit history, following the submission rules, honest reporting, a README that works on a clean machine.

Shortlisted candidates come to a 90-minute session where we work through a small real task together in one of our actual repositories. Nothing to prepare for it.

Good luck — and please do email if anything here does not make sense.